

Data Mining for Business Analytics

MSBA 511

k-Nearest Neighbors (KNN)

k-Nearest Neighbors Overview

The k-Nearest Neighbors (KNN) algorithm is another supervised learning method used for classification and regression tasks.

Basic idea: Classify a record like similar records

Some key points:

- It is **data driven**, not model driven.
 - An explicit model is not created during training so is considered a "lazy learner" since computation is postponed until needed.
- Performance is highly dependent on the choice of **k**.
- Relies on a **distance metric** and classifies new data points based on their similarity to existing observations in the training data.
- **Non-parametric** so it makes no assumptions about the data distribution.
- Data must be **normalized** since it can be heavily influenced by the scaling of the input features.

How to Measure "Nearby"

Like k-Means clustering, kNN relies on a distance metric to determine the similarity between data points.

Minkowski is a more generic distance metric that is the equivalent of **Manhattan distance** where **p = 1** and **Euclidean distance** where **p = 2**.

Minkowski Distance (Generalized Form)

$$d(x, y) = \left(\sum_{i=1}^n |x_i - y_i|^p \right)^{\frac{1}{p}} \text{ for } p \geq 1$$

Euclidean Distance

$$d(x, y) = \sqrt{\sum_{i=1}^n (x_i - y_i)^2}$$

Other distance measures, such as **cosine** similarity, can also be used depending on the nature of the data.



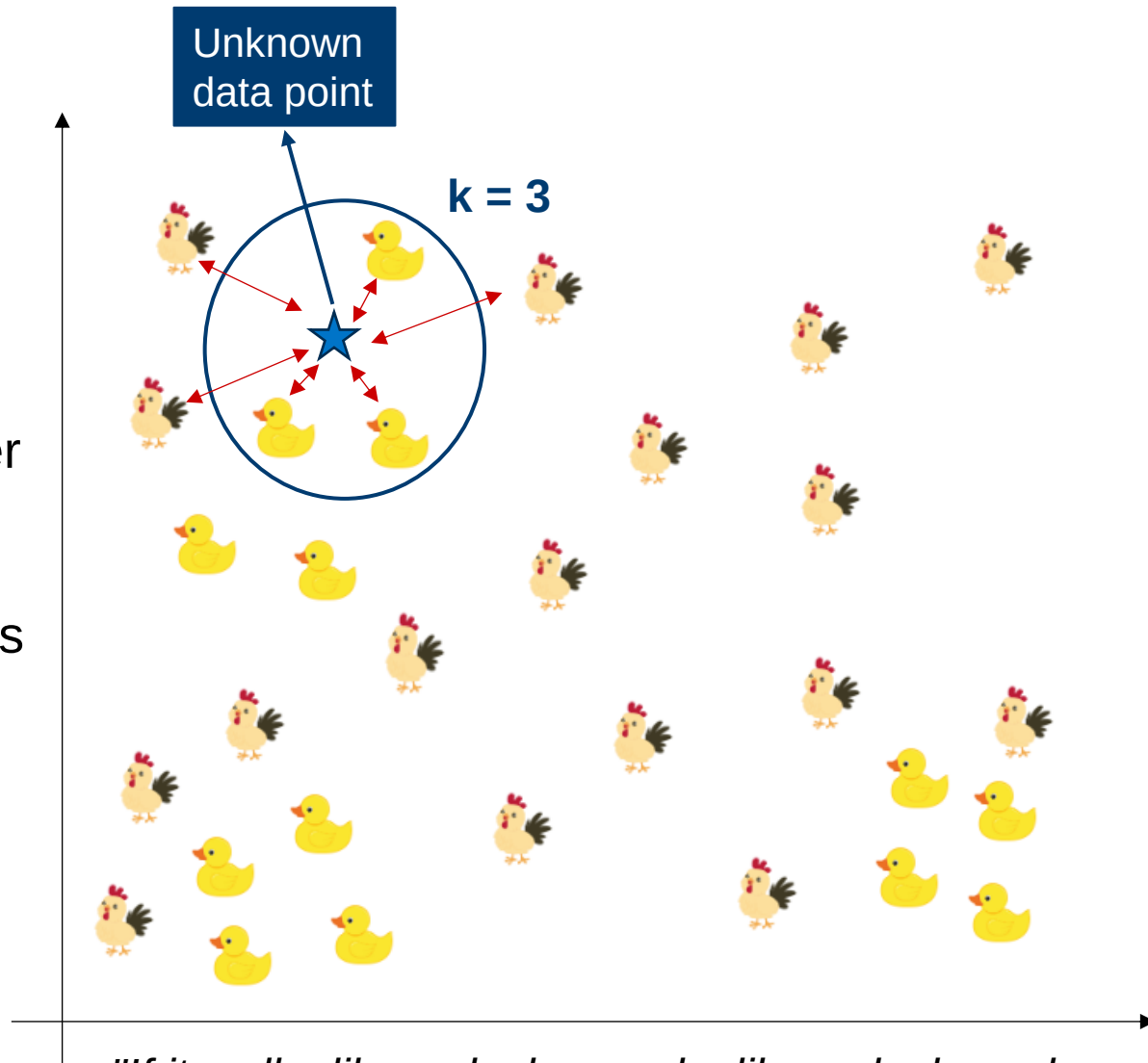
The sklearn implementation of KNeighborsClassifier utilizes the default minkowski metric with a power parameter of 2 (i.e., Euclidean distance).

Choosing k

k is the number of nearby neighbors.

To classify a new data point:

- Compute distances to other training records.
- Identify k nearest neighbors
- Use class labels to label new data point based on majority vote of its k nearest neighbors.



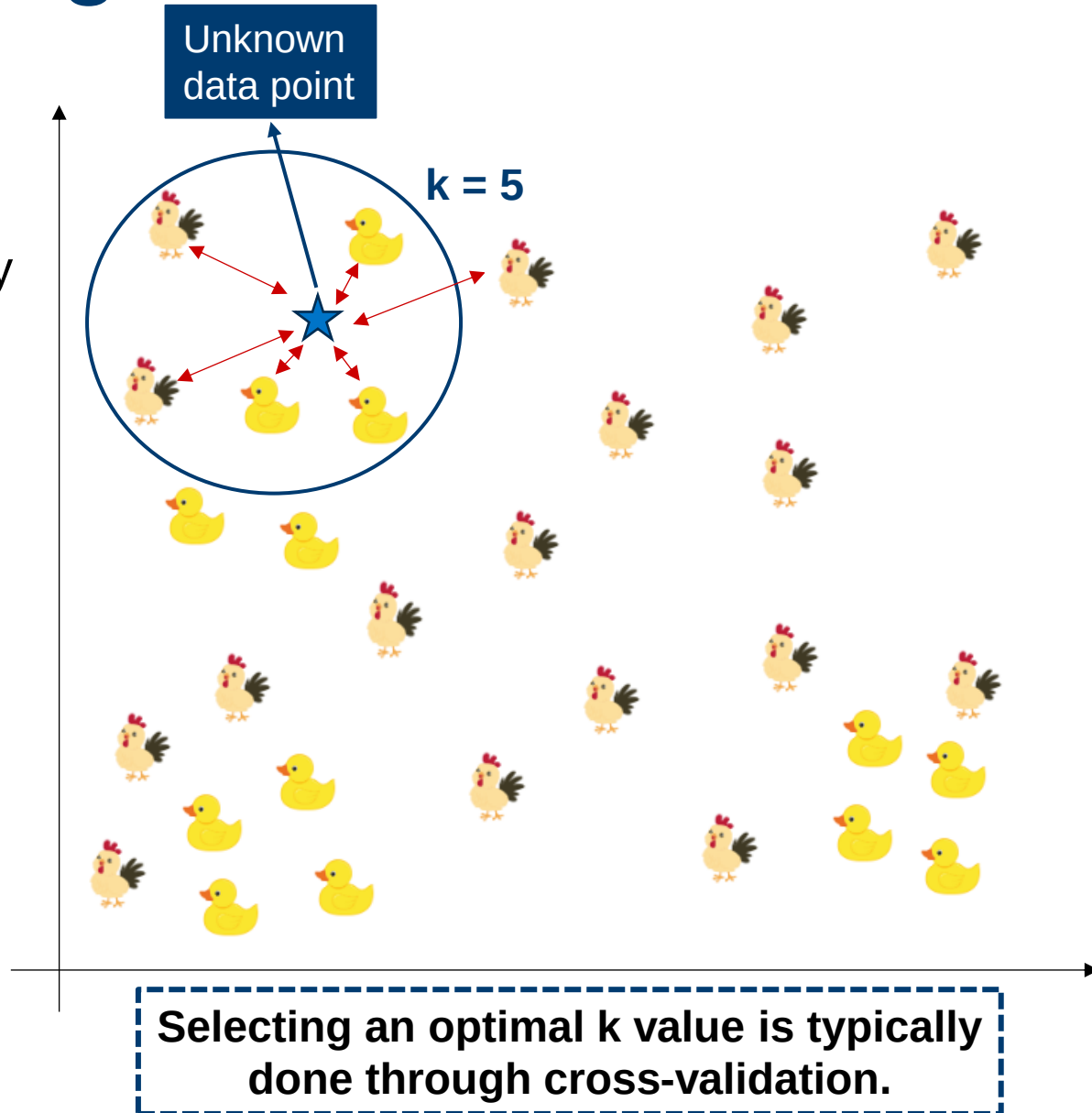
"If it walks like a duck, quacks like a duck, and looks like a duck, then it's probably a duck."

Low k vs. High k

Low values of k (1, 3, ..) captures local structure in data but also noise potentially leading to overfitting.

High values of k can smooth out decision boundaries, improving generalization, but may miss local structure.

The extreme case of $k = n$ is the same as the "naive rule" classify all records according to majority class.



Riding Mowers Example Revisited

Income	Lot Size	Ownership
60.0	18.4	owner
85.5	16.8	owner
64.8	21.6	owner
61.5	20.8	owner
87.0	23.6	owner
110.1	19.2	owner
108.0	17.6	owner
82.8	22.4	owner
69.0	20.0	owner
93.0	20.8	owner
51.0	22.0	owner
81.0	20.0	owner
75.0	19.6	non-owner
52.8	20.8	non-owner
64.8	17.2	non-owner
43.2	20.4	non-owner
84.0	17.6	non-owner
49.2	17.6	non-owner
59.4	16.0	non-owner
66.0	18.4	non-owner
47.4	16.4	non-owner
33.0	18.8	non-owner
51.0	14.0	non-owner
63.0	14.8	non-owner

A company sells riding lawn mowers and is interested in **classifying** families in a city that are likely to purchase a riding mower. We have data for 24 households where **Ownership** of a mower is labeled. We have **predictor** variables **Income** and **Lot Size** to predict ownership.

Split and Normalize Data

Split the data into train and test partitions and normalize the entire dataset using the train data only.

60% Train Partition

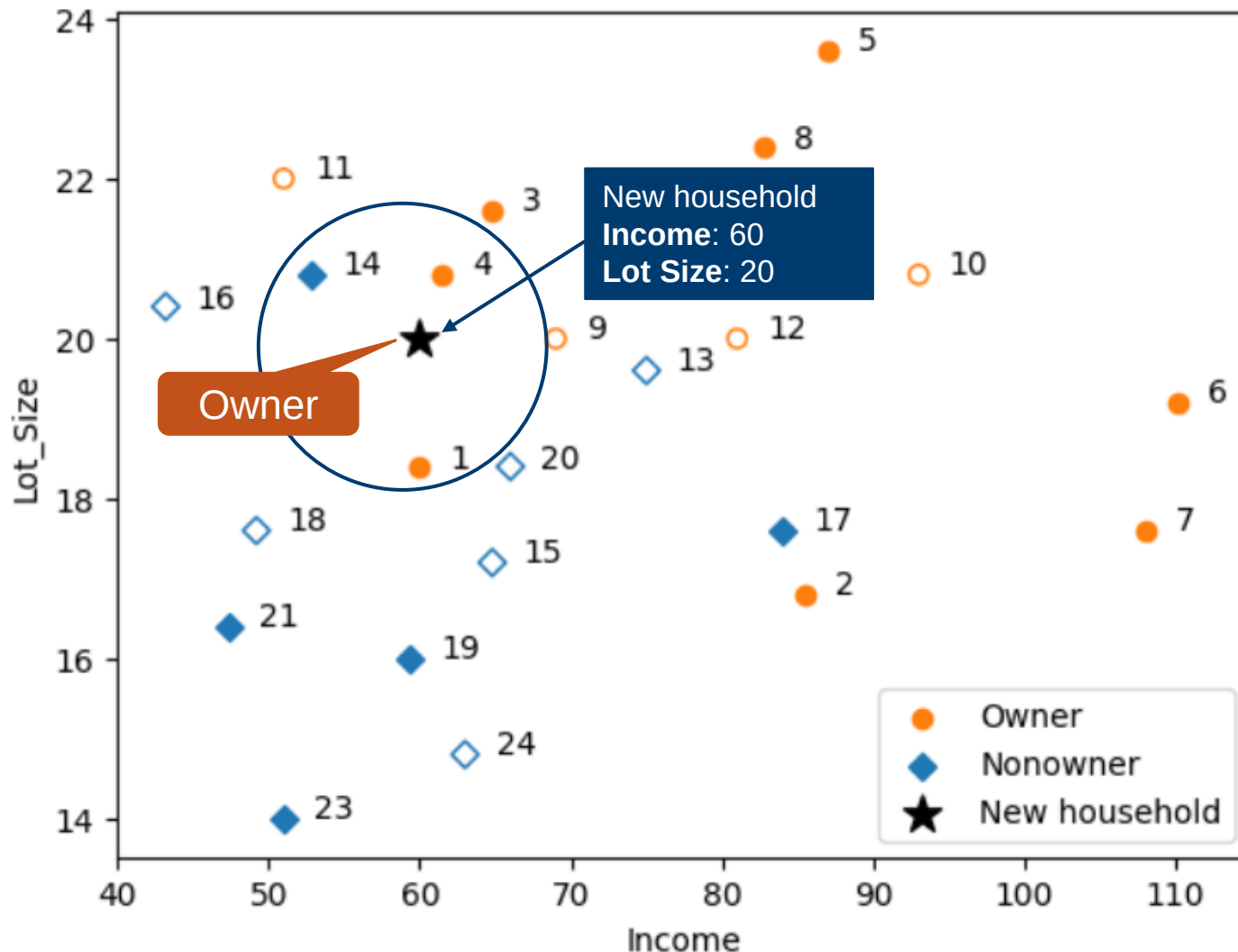
	Income	Lot_Size	Ownership	zIncome	zLot_Size
5	110.1	19.2	Owner	1.797760	0.131181
3	61.5	20.8	Owner	-0.409776	0.743358
7	82.8	22.4	Owner	0.557724	1.355535
20	47.4	16.4	Nonowner	-1.050235	-0.940129
18	59.4	16.0	Nonowner	-0.505164	-1.093174
2	64.8	21.6	Owner	-0.259882	1.049447
4	87.0	23.6	Owner	0.748499	1.814668
0	60.0	18.4	Owner	-0.477910	-0.174908
13	52.8	20.8	Nonowner	-0.804953	0.743358
22	51.0	14.0	Nonowner	-0.886713	-1.858395
1	85.5	16.8	Owner	0.680365	-0.787085
16	84.0	17.6	Nonowner	0.612231	-0.480996
6	108.0	17.6	Owner	1.702373	-0.480996
21	33.0	18.8	Nonowner	-1.704320	-0.021863

40% Test Partition

	Income	Lot_Size	Ownership	zIncome	zLot_Size
12	75.0	19.6	Nonowner	0.203428	0.284225
14	64.8	17.2	Nonowner	-0.259882	-0.634041
11	81.0	20.0	Owner	0.475964	0.437269
8	69.0	20.0	Owner	-0.069107	0.437269
15	43.2	20.4	Nonowner	-1.241009	0.590314
10	51.0	22.0	Owner	-0.886713	1.202491
17	49.2	17.6	Nonowner	-0.968474	-0.480996
19	66.0	18.4	Nonowner	-0.205375	-0.174908
23	63.0	14.8	Nonowner	-0.341643	-1.552307
9	93.0	20.8	Owner	1.021034	0.743358

Using the testing data to normalize the data is an example of data leakage.

Classify a New Household



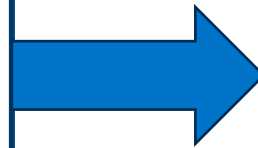
Step 1: Find the nearest neighbors for $k = 3$.

Step 2: Classify the new household based on the majority class for its neighbors.

Evaluation for Different k

Is **k = 3** the optimal value?

Calculate accuracy on different **k** values.



k	accuracy
1	0.6
2	0.7
3	0.8
4	0.9
5	0.7
6	0.9
7	0.9
8	0.9
9	0.9
10	0.8
11	0.8
12	0.9
13	0.4
14	0.4

Maximum accuracy with smallest k.

Odd values for k are typically used to avoid ties in voting.

Advantages

- Simple and works well with sufficiently large and balanced training data.
 - Majority voting on imbalanced datasets can lead to biased predictions.
- No assumptions required about underlying data distribution (i.e., Normal).
- Effective at capturing complex interactions among variables without having to define a statistical model.

Shortcomings

- Required size of training set increases exponentially with number of predictors, p .
 - Expected distance to nearest neighbor increases with p
- In a large training set, computationally expensive and potentially prohibitive to find distances to all neighbors at time of prediction.
- Dealing with the “curse of dimensionality”:
 - ✓ Use dimensionality reduction techniques such as PCA.
 - ✓ Use alternative distance metrics such as cosine similarity if the magnitude of the vectors does not matter.
 - ✓ Use alternative techniques such as Approximate Nearest Neighbors (ANN).